

EduConnect — Developer Handover Documentation

This document describes the **server** (Express + Prisma) and **web client** (React + Vite). The mobile workspace exists in `mobile/` but is intentionally out of scope here.

1. Overview

EduConnect is an in-app education marketplace connecting international students with Australian universities, education agents, and consultants. The core feature surface is:

- **Authentication & roles** — five roles: `ADMIN`, `UNIVERSITY`, `AGENT`, `CONSULTANT`, `STUDENT`. Role discriminates the User row in a single table; each non-admin role has a 1:1 profile table.
 - **Marketplace directories** — public listings for universities, agents, consultants. Includes a "compare universities" batch lookup.
 - **Posts / community feed** — categorised posts with upvotes, bookmarks, and threaded comments. Authors are users of any role.
 - **Bookings** — 1:1 meetings between any non-admin user and an agent / consultant / university provider. Statuses: `PENDING` → `CONFIRMED` → `COMPLETED` / `CANCELLED`.
 - **Leads** — student → university enquiry. Status flow: `NEW` → `CONTACTED` → `CONVERTED` / `CLOSED`.
 - **Reviews** — students post star + body reviews on a target user (uni / agent / consultant). Each review recomputes the target's `rating` and `reviewCount`.
 - **Campaigns** — universities own marketing campaigns with start/end dates and `ACTIVE/PAUSED/ENDED` status.
 - **In-app messaging** — 1:1 conversations with persistent history and live delivery via Socket.io.
 - **In-app meetings** — LiveKit rooms tied 1:1 to bookings; in-app token mint with time-window guards.
 - **Admin panel** — list users, approve / suspend / reactivate non-student accounts, pin / hide / remove posts, dashboard counts.
-

2. Tech stack

Server (`server/`)

- **Runtime:** Node.js 20+, ES modules, `node --watch` for dev.
- **Framework:** Express.
- **ORM / DB:** Prisma 6 + PostgreSQL.
- **Auth:** JWT access + opaque rotating refresh tokens (httpOnly signed cookies for web; `Authorization: Bearer` for mobile).
- **Password hashing:** Argon2id (OWASP-recommended params).
- **Validation:** zod schemas applied via a `validate` middleware.
- **Security:** Helmet (CSP, HSTS, frameguard, noSniff), CORS allow-list, CSRF (double-submit via `csrf-csrf`), `hpp` (HTTP parameter pollution), rate limits (`express-rate-limit`).
- **Real-time:** `socket.io` attached to the same HTTP server; auth handshake via signed cookie or Bearer token.
- **Video / audio:** `livekit-server-sdk` for token minting.
- **Logging:** Pino + `pino-http`; pretty-print in dev, redaction of secrets.
- **Cron:** `node-cron` for background jobs.

Client (`client/`)

- **Framework:** React 19 + Vite + Tailwind CSS v4.
- **Routing:** `react-router-dom` v6 with lazy-loaded routes.
- **State:** Zustand (auth-only; all domain data is fetched per-page).
- **HTTP:** custom fetch wrapper with CSRF token caching, silent refresh on 401, CSRF rotation on 403.
- **Real-time:** `socket.io-client`.
- **Video:** `@livekit/components-react` + `livekit-client` with custom focus-swap layout.
- **UI bits:** `framer-motion`, `lucide-react`, `recharts`, `react-hot-toast`.

3. Repository layout

client/	React + Vite SPA
server/	Express API + Socket.io gateway + Prisma
mobile/	Expo React Native (out of scope here)
prisma/	(lives under server/) schema + migrations + seed
docs/	handover docs
README.md	first-run setup

Server source layout

server/src/	
app.js	Express app builder (middleware order, /api mount, error handlers)
server.js	HTTP server boot, Socket.io attach, cron start, graceful shutdown
config/	
env.js	zod-validated env schema (fails boot on missing vars)
logger.js	Pino logger with redaction
db/	
prisma.js	singleton PrismaClient
jobs/	
cron.js	expired refresh-token cleanup, campaign auto-end, booking auto-
complete	
modules/	feature folders – one per domain
index.js	aggregates module routers under /api
auth/	auth.{controller,routes,schema,service}.js
posts/	posts.{controller,routes}.js + post.schema.js
directory/	directory.{controller,routes}.js + profile.schema.js
business/	campaigns/bookings/leads/reviews controllers + business.
{routes,schema}.js	
meetings/	meetings.{controller,routes,service}.js (LiveKit token endpoint)
messaging/	messaging.{controller,routes,schema,service,gateway}.js (REST +
Socket.io)	
admin/	admin.{controller,routes}.js
shared/	cross-cutting concerns
middleware/	auth, csrf, cors, errorHandler, securityHeaders, rateLimits, validate
utils/	tokens, cookies, asyncHandler, password, errors
validators/	common.schema (idParam, paginationQuery, emptyToUndef helper)

Client source layout

client/src/	
App.jsx	Routes, ProtectedRoute, RequireAuth, RedirectIfAuthenticated wrappers
api/	
client.js	fetch wrapper (CSRF, refresh, error mapping)
endpoints.js	typed API surface – one object per server module
mappers.js	UPPER_SNAKE ↔ lower-kebab + nested→flat normalisers
store/	
useStore.js	auth-only Zustand store (currentUser, isAuthenticated, hydrate, login,

```

logout)
hooks/
  useApiResource.js    generic data-fetching hook with refetch + optimistic-friendly setData
lib/
  socket.js           Socket.io singleton (cookie auth)
components/
  layout/             DashboardLayout, Navbar, Sidebar, Footer
  booking/            BookingModal
  meeting/            JoinMeetingButton
  notifications/      GlobalNotifications (meeting:started toast)
  messaging/          (per-message bubble in MessagesPage)
pages/               routed screens, grouped by role/feature
  auth/               Login, Signup
  admin/, university/, agent/, consultant/, student/
  marketplace/        Universities, Agents, Consultants, *Detail
  feed/, messages/, meeting/, dashboard/MyPosts/, static/

```

4. Environment configuration

Validated in `server/src/config/env.js` (zod). The server **fails to boot** if any required value is missing or malformed.

Variable	Type / format	Purpose
NODE_ENV	development test production (default development)	Toggles dev-only behaviour (pretty logs, looser cookie SameSite, error stack)
PORT	int (default 5000)	HTTP listener port; project conventionally uses 5050
CLIENT_URL	URL	Allow-listed origin for CORS and Socket.io
TRUST_PROXY	true / false (default false)	Set <code>app.set('trust proxy', 1)</code> so <code>req.ip</code> works behind nginx/ALB
DATABASE_URL	string ≥ 10 chars	Postgres connection URL
COOKIE_SECRET	string ≥ 32 chars	Signs the access/refresh cookies
JWT_ACCESS_SECRET	string ≥ 32 chars	HS256 signing key for access tokens
JWT_REFRESH_SECRET	string ≥ 32 chars	(Reserved; refresh tokens are opaque + hashed.)
CSRF_SECRET	string ≥ 32 chars	Seeds <code>csrf-csrf</code> HMAC
ACCESS_TOKEN_TTL	duration string (default 15m)	Access JWT lifetime
REFRESH_TOKEN_TTL_DAYS	int (default 30)	Refresh token lifetime
RATE_LIMIT_WINDOW_MS	int (default 900000)	API rate-limit window
RATE_LIMIT_MAX	int (default 300)	API rate-limit max per window
AUTH_RATE_LIMIT_MAX	int (default 10)	Login/refresh max failed attempts per window
LOG_LEVEL	trace-fatal (default info)	Pino log level
LIVEKIT_URL	URL starting with <code>wss://</code>	LiveKit WebSocket server URL

LIVEKIT_API_KEY	string ≥ 8 chars	LiveKit API key
LIVEKIT_API_SECRET	string ≥ 16 chars	LiveKit API secret (also signs webhooks)

Server `.env.example` is committed; copy to `.env` and fill in. Frontend has no `.env` requirements — it talks to `/api` same-origin in dev (Vite proxy).

5. Database schema (Prisma)

Source of truth: `server/prisma/schema.prisma`.

Enums

```

Role:          ADMIN, UNIVERSITY, AGENT, CONSULTANT, STUDENT
AccountStatus: PENDING, ACTIVE, SUSPENDED
Tier:          FREE, PREMIUM, ENTERPRISE
PostStatus:    PUBLISHED, HIDDEN, REMOVED
PostCategory:  SCHOLARSHIPS, VISA_TIPS, COURSES, CAMPUS_LIFE, CAREER, STUDENT_LIFE, EVENTS
MediaType:     NONE, IMAGE, VIDEO
BookingStatus: PENDING, CONFIRMED, COMPLETED, CANCELLED
LeadStatus:    NEW, CONTACTED, CONVERTED, CLOSED
CampaignStatus: DRAFT, ACTIVE, PAUSED, ENDED

```

Core identity

- **User** — single users table with `role` discriminator. Holds `email`, `passwordHash` (argon2id), `name`, `avatarUrl`, `status` (defaults PENDING for non-students), `lastLoginAt`. 1:1 to one of `UniversityProfile` / `AgentProfile` / `ConsultantProfile` / `StudentProfile`.
- **UniversityProfile** — `userId @id`, `shortName`, `location`, `type`, `description`, `website`, `phone`, `logoUrl`, `coverImageUrl`, `foundedYear`, `studentCount`, `internationalPct`, `ranking`, `rating`, `reviewCount`, `views`, `inquiries`, `tier`, `verified`, `tuitionMin / Max / Currency`, plus string arrays `courses`, `scholarships`, `intakes`, `facilities`, `accreditations`.
- **AgentProfile** — `contactPerson`, `phone`, `location`, `description`, `website`, `logoUrl`, `yearsExperience`, `studentsPlaced`, `partnerInstitutions`, `successRate`, `rating`, `reviewCount`, `verified`, `tier`, arrays `certifications` / `services` / `languages` / `specialisations`, `maraNumber`.
- **ConsultantProfile** — `phone`, `location`, `description`, `website`, `yearsExperience`, `studentsAssisted`, `successRate`, `rating`, `reviewCount`, `verified`, `tier`, `qualifications` / `services` / `languages` / `specialisations`, `hourlyRate`.
- **StudentProfile** — `phone`, `nationality`, `currentEducation`, `interestedIn`, `preferredLocations`, `budgetMin / Max`, `bio`, `intakeTarget`.

Auth

- **RefreshToken** — `tokenHash` (sha256 of opaque token), `userId`, `userAgent`, `ip`, `expiresAt`, `revokedAt`, `replacedBy` (chain for rotation).

Posts / social

- **Post** — `authorId`, `title`, `content`, `category`, `mediaType`, `mediaUrl`, `tags[]`, `status` (default PUBLISHED), `isPinned`, `upvoteCount`, `commentCount`, `timestamps`.
- **Comment** — `postId`, `authorId`, `text`, `upvoteCount`, `timestamps`.
- **PostVote** — composite key (`userId`, `postId`).
- **Bookmark** — composite key (`userId`, `postId`).

Reviews

- **Review** — `reviewerId`, `targetId`, `targetRole` (UNIVERSITY/AGENT/CONSULTANT), `rating` (1-5), `title?`, `body`, `verified`. Unique on (`reviewerId`, `targetId`).

Bookings + meetings

- **Booking** — `studentId` (the BOOKER, regardless of role), `providerId`, `subject`, `notes`, `scheduledAt`, `durationMinutes` (default 30), `mode` (`video` / `phone` / `in-person`, default `video`), `status` (default PENDING). 1:1 to `Meeting`.
- **Meeting** — `bookingId` @unique, `roomName` @unique (random UUID, never the bookingId), `maxParticipants` (default 2), `startedAt`, `endedAt`, timestamps.

Leads

- **Lead** — `studentId`, `universityId`, `programme?`, `message`, `status` (default NEW), timestamps.

Campaigns

- **Campaign** — `universityId`, `name`, `audience`, `startDate`, `endDate`, `status` (default DRAFT), `impressions`, `clicks`.

Messaging

- **Conversation** — `userId` and `userBId` stored in **canonical sorted order** so (`userId` < `userBId`) and the pair is unique. Holds `lastMessageAt` for ordering.
- **Message** — `conversationId`, `senderId`, `body`, `createdAt`. Indexed on (`conversationId`, `createdAt`).
- **ConversationRead** — composite (`conversationId`, `userId`) cursor with `lastReadAt`. Used to compute unread counts.

Audit log

- **AuditLog** — `actorId?`, `action`, `entityType`, `entityId?`, `meta: Json?`, `ip`, `userAgent`, timestamps. Used for moderation + meeting-token issuance trail.

6. Authentication & security

Token model

- **Access token:** JWT signed HS256 with `JWT_ACCESS_SECRET`. TTL = `ACCESS_TOKEN_TTL` (default 15m). Payload: { `sub: userId`, `role` }.
- **Refresh token: opaque** 64-byte random hex string (NOT a JWT). The DB stores only the SHA-256 hash. Rotated on every refresh; reuse of a revoked token revokes the entire family for that user (theft heuristic).

Where tokens travel

- **Web SPA:** httpOnly **signed** cookies — `em_access` (path /) and `em_refresh` (path /api/auth). SameSite `strict` in prod, `lax` in dev. Cookie auth requires CSRF.
- **Mobile / API clients:** Authorization: Bearer <accessToken> header. Bearer-authenticated requests are immune to CSRF (browsers can't forge headers via a CSRF attack), so the server skips CSRF for them.

`attachUser` middleware (`shared/middleware/auth.js`)

Bearer header takes precedence over the cookie. If `Authorization: Bearer ...` is present, the request is marked `isBearerAuth = true` and CSRF is skipped. Otherwise the signed cookie is consulted.

CSRF (`shared/middleware/csrf.js`)

- Library: `csrf-csrf` (double-submit cookie pattern).
- Cookie name: `__Host-em.csrf` in prod, `em.csrf` in dev.

- Token header on mutating requests: `X-CSRF-Token` .
- Issued via `GET /api/auth/csrf` (idempotent; force-overwrites stale cookies).
- Skipped automatically for Bearer-authed requests.

Helmet / extra headers

Helmet configures CSP (`default-src 'none'` , `connect-src 'self'` , `frame-ancestors 'none'`), HSTS (1y, includeSubDomains, preload), `X-Frame-Options DENY` , `X-Content-Type-Options nosniff` , `Cross-Origin-Resource-Policy same-site` , `Referrer-Policy no-referrer` . Plus a manual `Permissions-Policy` and `Cache-Control: no-store` .

Rate limits (`shared/middleware/rateLimits.js`)

- **API limiter:** mounted on `/api` . Window `RATE_LIMIT_WINDOW_MS` , max `RATE_LIMIT_MAX` . Key by user id (falls back to IP).
- **Auth limiter:** login + refresh. `skipSuccessfulRequests: true` so only failed attempts count.
- **Signup limiter:** 5 / hour, IP-based.
- **Token mint limiter:** 10 / minute / user (in `meetings.routes.js`).
- **Send-message limiter:** 60 / minute / user (in `messaging.routes.js`).

Other guards

- `hpp` on every request (HTTP parameter pollution).
- `compression` , `cookie-parser` with `COOKIE_SECRET` , body limit `100kb` .
- Pino redacts `Authorization` , `Cookie` , `Set-Cookie` , `*.password` , `*.passwordHash` , `*.token` , `*.accessToken` , `*.refreshToken` , `*.LIVEKIT_API_SECRET` , `*.apiSecret` .

7. REST API reference

All paths are prefixed with `/api` . JSON in / JSON out.

Conventions

- **Auth (cookie):** the signed `em_access` cookie is sent automatically by the browser. The web client uses `credentials: 'include'` .
- **Auth (Bearer):** include header `Authorization: Bearer <accessToken>` .
- **CSRF:** for cookie-authed mutating calls (POST/PUT/PATCH/DELETE) include header `X-CSRF-Token: <token>` . Get the token from `GET /api/auth/csrf` . Bearer-authed calls skip CSRF.
- **Pagination:** `?page=1&limit=20` (max 50–100 depending on the route).
- **Errors:** `{ error: { code, message, details? } }` . `details.issues[]` is populated for zod validation failures.

7.1 Health

- `GET /api/health` — unrated. Returns `{ server, uptime, timestamp, memory, database }` . Returns 503 when Postgres is unreachable.

7.2 Auth (`/api/auth`)

Method & path	Auth	CSRF	Body / query	Description
<code>GET /csrf</code>	Public	—	—	Issue a CSRF token (also sets the cookie). Force-overwrites.
<code>POST /signup</code>	Public	No	<code>signupSchema</code> (discriminated by role)	Creates a user. STUDENT auto-active; UNI/AGENT/CONSULTANT start PENDING. Sets cookies + returns <code>{ user, tokens }</code> .

POST /login	Public	No	{ email, password }	Verifies argon2id hash. Constant-time on missing user. Returns { user, tokens } and sets cookies. Updates lastLoginAt.
POST /refresh	Public	No	{ refreshToken? } (web reads cookie)	Rotates the refresh token; detects reuse and revokes the family.
POST /logout	Required	Yes	{ refreshToken? }	Revokes the presented refresh token, clears cookies.
GET /me	Optional	—	—	Returns { user } or { user: null }.

signupSchema requires email (≤ 254), strong password (≥ 8 with upper/lower/digit), name (2–120), role discriminator, and role-specific optional fields (uni: shortName/location/type/description/website/phone; agent: contactPerson/phone/location/description/maraNumber/yearsExperience; consultant: phone/location/description/yearsExperience/hourlyRate; student: phone/nationality/currentEducation/interestedIn/preferredLocations/budgetMin/Max).

7.3 Directory (/api/{universities,agents,consultants,students})

Public for listings; profile updates require the matching role.

Method & path	Auth	Role	Body	Description
GET /universities	Public	—	listQuery	Paginated list. Filters: q, verified, location. Sort: rating (default), name, newest.
GET /universities/compare?ids=a,b,c	Public	—	—	Batch lookup, returns 2–4 IDs preserving caller order.
GET /universities/:id	Public	—	—	Detail (also bumps views async).
PATCH /universities/me	Required	UNIVERSITY	updateUniversitySchema	Update own University profile.
GET /agents, GET /agents/:id, PATCH /agents/me	...	AGENT	updateAgentSchema	Same shape as universities.
GET /consultants, GET /consultants/:id, PATCH /consultants/me	...	CONSULTANT	updateConsultantSchema	Same shape.
PATCH /students/me	Required	STUDENT	updateStudentSchema	Self-update only — no public student listing.

7.4 Posts (/api/posts)

Method & path	Auth	Body / query	Description
GET /	Optional	postListQuery	Sorted by hot (default — pinned, upvotes, recent), new, or top. Includes per-viewer hasUpvoted / hasBookmarked.
GET /me	Required	—	Posts authored by the caller.

GET /bookmarks	Required	—	Posts the caller has bookmarked.
GET /:id	Optional	—	Post + comments + viewer flags.
POST /	Required	createPostSchema	Create. Author = caller.
PATCH /:id	Required	updatePostSchema	Update. Author or ADMIN only.
DELETE /:id	Required	—	Author or ADMIN only.
POST /:id/upvote	Required	—	Toggle upvote (transactional with upvoteCount).
POST /:id/bookmark	Required	—	Toggle bookmark.
GET /:id/comments	Public	—	List comments.
POST /:id/comments	Required	createCommentSchema	Add comment + bump commentCount.
DELETE /comments/:id	Required	—	Author or ADMIN only. Decrements commentCount.

createPostSchema : title (3-200), content (10-10000), category (enum), mediaType (default NONE), mediaUrl? (URL), tags[] (max 5).

7.5 Bookings + business (/api)

These all live in the business module router and are mounted at the root of /api (no extra prefix). Bookings allow any non-admin role as the booker, and AGENT / CONSULTANT / UNIVERSITY as the provider.

Method & path	Auth	Role	Body
GET /campaigns	Required	UNIVERSITY	—
POST /campaigns	Required	UNIVERSITY	createCampaignSchema
PATCH /campaigns/:id	Required	UNIVERSITY	updateCampaignSchema
DELETE /campaigns/:id	Required	UNIVERSITY	—
GET /bookings	Required	—	—
POST /bookings	Required	STUDENT/AGENT/CONSULTANT/UNIVERSITY	createBookingSchema
PATCH /bookings/:id/status	Required	—	updateBookingStatusSchema
GET /leads	Required	UNIVERSITY	—
GET /leads/mine	Required	STUDENT	—
POST /leads	Required	STUDENT	createLeadSchema

PATCH /leads/:id/status	Required	UNIVERSITY	updateLeadStatusSchema
GET /reviews/target/:id	Public	—	—
POST /reviews	Required	STUDENT	createReviewBodySchema

Booking validation: `subject` (3-200), `scheduledAt` (date), `durationMinutes` (10-240, default 30), `mode` (`video` / `phone` / `in-person` , default `video`), `notes?` .

7.6 Meetings (/api)

Method & path	Auth	CSRF	Body	Description
POST /bookings/:id/meeting/token	Required	Yes	—	Mints a 15-min LiveKit JWT scoped to the meeting's room. Rate-limit 10/min/user.

Server-side guards (in `meetings.service.js`):

1. Caller must be the booker or the provider on the booking.
2. Booking must be `CONFIRMED` .
3. Booking `mode` must be `video` or `phone` (`in-person` rejected).
4. Current time must be within `scheduledAt - 15 min` and `scheduledAt + duration + 15 min` .
5. The user must not already have another active meeting in a still-open join window (single concurrent session).
6. `Meeting.startedAt` is stamped on the first token issuance and a `meeting:started` event is emitted via Socket.io to the other party's personal room.
7. Every issuance is appended to `AuditLog` with `action: meeting.token_issued` , `meetingId` , `userId` , `ip` , `userAgent` .

Token grant: `roomJoin` , `canPublish` , `canSubscribe` , `canPublishData` . Identity is `userId` ; display name is `user.name` . TTL 15 min.

Response: `{ token, wsUrl, roomName, meetingId, mode, expiresIn }` .

7.7 Messaging (/api/messages)

All endpoints require auth. Anyone can message anyone (other than themselves). Suspended users are not addressable.

Method & path	CSRF	Body / query	Description
GET /conversations	—	—	Caller's conversations sorted by <code>lastMessageAt</code> , with last-message preview and unread count.
POST /conversations	Yes	<code>{ userId }</code>	Get-or-create the conversation with <code>userId</code> . Pairs are stored in canonical sorted order (one row per pair).
GET /conversations/:id	—	—	Single conversation header.
GET /conversations/:id/messages?before=&limit=	—	<code>listMessagesQuery</code>	Cursor-paginated descending; client reverses for display.

POST /conversations/:id/messages	Yes	{ body } (1–4000)	Persists, bumps lastMessageAt, broadcasts via Socket.io. Rate-limit 60/min/user.
POST /conversations/:id/read	Yes	—	Upserts the caller's ConversationRead.lastReadAt to now.

7.8 Admin (/api/admin)

All endpoints require AUTH + role ADMIN.

Method & path	Body / query	Description
GET /overview	—	{ counts: { universities, agents, consultants, students, posts, pendingApprovals, bookings, leads } }
GET /users	?role=&status=&q=&page=&limit=	Paginated users with profiles.
POST /users/:id/approve	—	Sets status ACTIVE and verified=true on the profile. UNI/AGENT/CONSULTANT only.
POST /users/:id/suspend	—	Sets status SUSPENDED and revokes all refresh tokens for the user.
POST /users/:id/reactivate	—	Sets status ACTIVE.
POST /posts/:id/pin	—	Toggle isPinned.
PATCH /posts/:id/status	{ status: PUBLISHED HIDDEN REMOVED }	Set post status.

8. Real-time / Socket.io

Attached to the same HTTP server. Path: /socket.io . CORS uses CLIENT_URL .

Authentication

io.use runs on every connection. Resolves the user by either:

1. Bearer token sent via socket.handshake.auth.token (mobile / API clients), or
2. The signed em_access cookie parsed identically to Express (web).

Suspended users are rejected. Failed handshakes emit a Socket auth rejected log line.

Rooms

Each socket joins:

- user:<userId> — personal room used for cross-tab notifications (new message, meeting started).
- conv:<conversationId> — joined explicitly via conversation:join (server re-checks participation against the DB).

Events

Direction	Event	Payload	Notes
-----------	-------	---------	-------

client → server	conversation:join	(conversationId, ack)	Server validates participation, joins the room, calls ack { ok: true } or { ok: false, error }.
client → server	conversation:leave	(conversationId)	No ack.
server → client	message:new	{ conversationId, message }	Emitted to conv:<id> AND both participants' user:<id> rooms.
server → client	meeting:started	{ bookingId, meetingId, startedBy: { id, name }, mode, subject }	Emitted only to the OTHER party's user:<id> room when the first token is minted.

The client should replay the personal room subscription on `connect` (Socket.io reconnects automatically).

9. Web client architecture

Routing (`App.jsx`)

Layered:

- `RedirectIfAuthenticated` for `/login` , `/signup` — bounces to `/` if already signed in.
- `RequireAuth` for marketplace, feed, post detail, `/messages`, `/meeting/:bookingId`, static pages.
- `ProtectedRoute` `allowedRoles=[...]` for the role dashboards (`/admin` , `/university` , `/agent` , `/consultant` , `/student`) — wraps `<DashboardLayout />` .

`<ScrollToTop />` resets scroll on every navigation. `<GlobalNotifications />` mounts at app root and shows a toast with a "Join now" button on `meeting:started` .

Auth + state (`store/useStore.js`)

Zustand store, **auth only**. Domain data is fetched per-page. Shape:

```
{
  currentUser, isAuthenticated, authReady,
  hydrate(), login(email, password), signup(payload), logout(), setCurrentUser(user)
}
```

`hydrate()` is invoked once on App mount (`/auth/me`). Roles are normalised lowercase for the existing UI (`admin` / `university` / `agent` / `consultant` / `student`).

API client (`api/client.js`)

A single fetch wrapper with:

- `credentials: 'include'` always (cookie auth).
- `X-CSRF-Token` header injected on mutating calls; token cached in module scope and refetched on demand.
- Silent refresh on 401 — calls `/auth/refresh` once and retries the original request.
- CSRF rotation on 403 + `code: CSRF_INVALID` — drops the cached token and retries once.
- Concurrent refresh / CSRF deduplication via promise singletons.
- Throws a typed `ApiError(message, { status, code, details })` .

`api.resetCsrf()` is called by login/signup/logout because the session identifier changes and the old CSRF token is no longer valid.

Endpoints (`api/endpoints.js`)

One object per server module: `authApi` , `directoryApi` , `postsApi` , `reviewsApi` , `campaignsApi` , `bookingsApi` , `meetingsApi` , `messagingApi` , `leadsApi` , `adminApi` . Mirrors the REST surface 1:1.

Mappers (`api/mappers.js`)

Bridges the **server** vocabulary (UPPER_SNAKE enums, nested `user.university / user.agent` /etc.) to the **UI** vocabulary (lower-kebab, flat shape):

- `ROLE_FROM_API / ROLE_TO_API`
- `CATEGORY_FROM_API / CATEGORY_TO_API`
- `MEDIA_FROM_API / MEDIA_TO_API`
- `BOOKING_STATUS_FROM_API` , `LEAD_STATUS_FROM_API` , `CAMPAIGN_STATUS_FROM_API`
- `normalisePost(p)` , `normaliseDirectoryItem(u)` , `normaliseBooking(b)` , `normaliseLead(l)` , `normaliseCampaign(c)` — pages call these after `useApiResponse(...)` .

Data fetching (`hooks/useApiResponse.js`)

Generic hook designed for React 19 / Strict Mode:

- `fetcher` stored in a ref → no closure-thrash on re-renders.
- `deps` hashed to a stable `depsKey` .
- Spinner only on the **initial** load — refetches keep stale data on screen so actions don't flash.
- Returns `{ data, loading, error, refetch, setData }` . `setData` is used for optimistic updates.

Realtime client (`lib/socket.js`)

Singleton `io()` connection initialised on first call. Reuses the cookie session for auth (mobile uses `auth.token` + `tokenStorage`).

Notable UI components

- **BookingModal** — subject, date, time, duration, mode toggle, notes. Validates client-side (subject ≥ 3 chars, future time, can't book self / admin). Surfaces server zod issues by `path: message` .
- **JoinMeetingButton** — mirrors the server's join-window. Renders nothing outside the window or for `in-person` bookings; shows "Opens in N min" before, "Join meeting" / "Start call" inside. Phone bookings use a `Phone` icon.
- **MeetingRoom** (in `pages/meeting/`) — fetches `meetingsApi.joinToken` , mounts `<LiveKitRoom>` , custom focus-swap layout (large remote + PiP local, click to swap), themed control bar, in-meeting `<Chat />` panel, "waiting for the other party" overlay, brand colour overrides via CSS variables (`livekitTheme`).
- **MessagesPage** — two-pane: conversation list (left) + thread (right). Marks read on open. Uses optimistic-insert for outgoing messages (temp id swapped for the server-returned message).
- **GlobalNotifications** — listens for `meeting:started` on the personal room, shows a toast with **Join now** / **Later**.

10. Background jobs (`jobs/cron.js`)

Three jobs. Timezone Australia/Melbourne.

Job	Schedule	Action
<code>cleanupRefreshTokens</code>	<code>10 2 * * *</code>	Deletes refresh tokens that are expired or revoked > 30 days.
<code>endExpiredCampaigns</code>	<code>0 * * * *</code>	<code>Campaign.status: ACTIVE</code> → <code>ENDED</code> where <code>endDate < now</code> .
<code>completePastBookings</code>	<code>30 * * * *</code>	<code>Booking.status: CONFIRMED</code> → <code>COMPLETED</code> where <code>scheduledAt < now - 1h</code> .

Started via `startCronJobs()` after `app.listen`. Stopped via `stopCronJobs()` during graceful shutdown (SIGINT / SIGTERM). The HTTP server then drains, Prisma disconnects, with a 10s force-exit fallback.

11. Validation, error handling, conventions

Zod schemas

Every mutating endpoint gets a `validate({ body | query | params })` middleware. Schemas live next to their module (`*.schema.js`). Errors are converted to a typed `BadRequestError` with `details.issues = [{ path, message }]` so the client can show field-level feedback.

Typed application errors (`shared/utils/errors.js`)

- `AppError(message, { status, code, details? })` — base class.
- `BadRequestError (400, BAD_REQUEST)`
- `UnauthorizedError (401, UNAUTHORIZED)`
- `ForbiddenError (403, FORBIDDEN)`
- `NotFoundError (404, NOT_FOUND)`
- `ConflictError (409, CONFLICT)`
- `TooManyRequestsError (429, RATE_LIMITED)`

Error middleware (`shared/middleware/errorHandler.js`)

Maps:

- CSRF rejection → 403 `CSRF_INVALID`
- `AppError` instance → its declared status + code
- Prisma `P2002` → 409 `CONFLICT`
- Prisma `P2025` → 404 `NOT_FOUND`
- CORS rejection (`Origin ... not allowed by CORS`) → 403 `CORS_BLOCKED`
- Anything else → 500 `INTERNAL` with stack only in non-prod

`notFoundHandler` returns 404 `NOT_FOUND` for unknown routes.

`asyncHandler`

Tiny wrapper used on every async route handler so thrown errors propagate to the error middleware:

```
asyncHandler(fn) = (req, res, next) => Promise.resolve(fn(...)).catch(next) .
```

Logging

- Pino + pino-http; pretty in dev.
- Auto-skip `/api/health` to avoid noise.
- Per-request: log level upgraded to `error` on 5xx, `warn` on 4xx.
- Redacted: `Authorization` , `Cookie` , `Set-Cookie` , password fields, JWTs / opaque tokens, LiveKit secret-like keys.

12. Operational notes

Database migrations

```
npm run migrate          # prisma migrate dev (creates a new migration)
npm run prisma:deploy    # in CI / prod (applies pending migrations)
npm run seed             # repopulates demo data
```

Graceful shutdown

`SIGINT` / `SIGTERM` triggers cron stop, HTTP drain, and `prisma.$disconnect()` . Force-exit timer at 10s.
Unhandled rejections / uncaught exceptions log fatal and `process.exit(1)` .

Crash boundaries

- `process.on('unhandledRejection')` → fatal log + exit.
- `process.on('uncaughtException')` → fatal log + exit.

Prisma client

Singleton in `db/prisma.js` . Hot-reload-safe via `globalThis.__prisma__` .

13. Directory of all files (quick index)

Server

```
src/  
  app.js, server.js  
  config/env.js, config/logger.js  
  db/prisma.js  
  jobs/cron.js  
  modules/  
    index.js  
    auth/{auth.controller, auth.routes, auth.schema, auth.service}.js  
    posts/{posts.controller, posts.routes, post.schema}.js  
    directory/{directory.controller, directory.routes, profile.schema}.js  
    business/{campaigns,bookings,leads,reviews}.controller.js  
      business.{routes,schema}.js  
    meetings/{meetings.controller, meetings.routes, meetings.service}.js  
    messaging/{messaging.controller, messaging.routes, messaging.schema,  
      messaging.service, messaging.gateway}.js  
    admin/{admin.controller, admin.routes}.js  
  shared/  
    middleware/{auth, csrf, cors, errorHandler, securityHeaders,  
      rateLimits, validate}.js  
    utils/{tokens, cookies, asyncHandler, password, errors}.js  
    validators/common.schema.js  
prisma/  
  schema.prisma, seed.js, migrations/*
```

Client

```
src/  
  App.jsx, main.jsx  
  api/{client, endpoints, mappers}.js  
  store/useStore.js  
  hooks/useApiResource.js  
  lib/socket.js  
  components/  
    layout/{DashboardLayout, Navbar, Sidebar, Footer}.jsx  
    booking/BookingModal.jsx  
    meeting/JoinMeetingButton.jsx  
    notifications/GlobalNotifications.jsx
```

```
pages/  
  auth/{Login, Signup}.jsx  
  feed/(via Feed.jsx, PostDetail.jsx)  
  marketplace/{Universities, Agents, Consultants,  
    UniversityDetail, AgentDetail}.jsx  
  admin/{AdminDashboard, ManageUniversities, ManageAgents,  
    ManageConsultants, ManagePosts, AdminSettings}.jsx  
  university/{UniDashboard, UniProfile, UniAnalytics,  
    UniCampaigns, UniBookings}.jsx  
  agent/{AgentDashboard, AgentProfile, AgentBookings,  
    AgentAnalytics}.jsx  
  consultant/{ConsultantDashboard, ConsultantProfile,  
    ConsultantBookings, ConsultantAnalytics}.jsx  
  student/{StudentDashboard, StudentProfile, StudentBookings}.jsx  
  dashboard/MyPosts.jsx  
  messages/MessagesPage.jsx  
  meeting/MeetingRoom.jsx (+ meeting.css)  
  static/* (about, contact, faq, blog, privacy, terms, ...)
```
